

Módulo I

Lógica de Programação

Portugol e VisualG

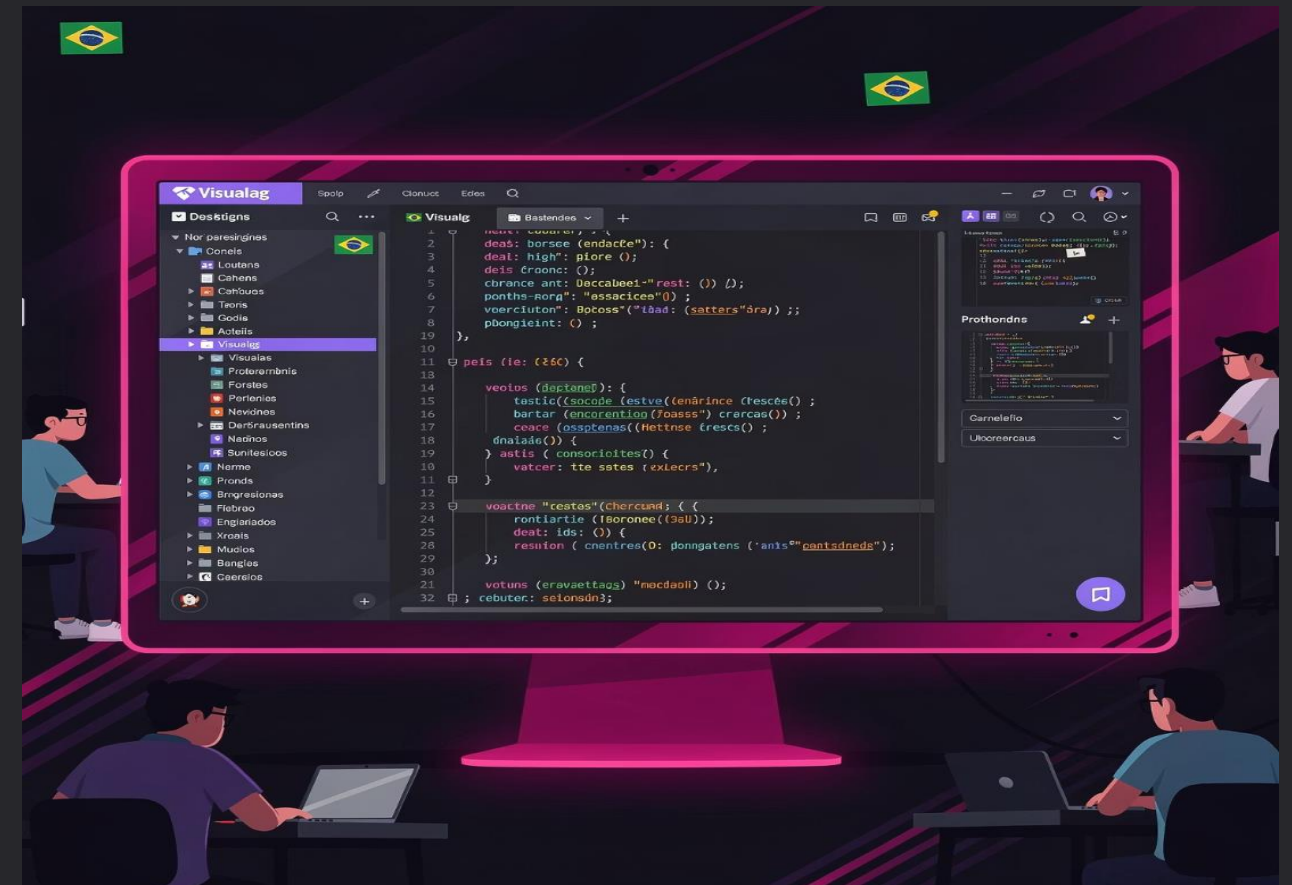


O que é Portugol?

Pseudolinguagem em Português

Portugol é uma pseudolinguagem criada para ensinar lógica de programação utilizando comandos escritos em português. Ao invés de lidar com inglês desde o início, o estudante pode focar na **lógica** e no raciocínio algorítmico.

No Brasil, é amplamente usado junto ao software **Visualg**, que permite escrever, executar e depurar algoritmos de forma simples, visual e gratuita.



Objetivo

Ensinar lógica antes de uma linguagem real

BR Idioma

Comandos totalmente em português

Estrutura Básica no Visualg

Todo programa escrito no Visualg segue uma estrutura padrão obrigatória. Compreender essa estrutura é o primeiro passo para escrever algoritmos corretos e organizados.

```
algoritmo
"nome_do_algoritmo"
var
    // declaração de
    variáveis
inicio
    // comandos do programa
finalgoritmo
```

O bloco **var** é onde você declara todas as variáveis antes de usá-las. O bloco entre **inicio** e **finalgoritmo** contém a lógica do seu programa.

01

algoritmo "nome"

Define o nome do programa — use aspas e sem espaços ou acentos

02

var

Seção de declaração de todas as variáveis que serão usadas

03

inicio ... finalgoritmo

Bloco principal onde os comandos são executados sequencialmente



Declaração de Variáveis

Variáveis são espaços na memória usados para armazenar informações durante a execução do programa. No Visualg, você deve declarar todas as variáveis na seção **var**, antes do **inicio**.

1 inteiro

Números sem casas decimais

Ex: `idade: inteiro`

10 real

Números com casas decimais

Ex: `altura: real`

T caractere

Textos e palavras

Ex: `nome: caractere`

logico

Verdadeiro ou Falso

Ex: `aprovado: logico`

```
var
  idade : inteiro
  nome  : caractere
  altura : real
  aprovado: logico
```


Entrada e Saída de Dados

Para que um programa seja útil, ele precisa receber informações do usuário e exibir resultados. No Visualg, isso é feito com os comandos **leia** e **escreva**.

Exemplo Prático

leia(variavel)

Pausa o programa e aguarda que o usuário digite um valor, armazenando-o na variável indicada.

```
leia(idade)
```

escreva("Texto")

Exibe uma mensagem ou valor na tela **sem** pular linha ao final.

```
escreva("Olá, mundo!")
```

escreval("Texto")

Igual ao **escreva**, mas **pula uma linha** após a exibição. Ideal para separar saídas.

```
escreval("Fim do programa")
```

```
escreva("Digite sua  
idade: ")  
leia(idade)  
escreval("Sua idade é:  
", idade)
```

Você pode combinar texto entre aspas e variáveis dentro do **escreva** separando-os por vírgula. Isso é muito útil para exibir mensagens personalizadas!

Operadores em Portugol

Os operadores são os símbolos que permitem realizar cálculos, comparações e tomar decisões lógicas dentro do seu algoritmo. Dominar esses operadores é essencial para construir qualquer programa.

+ Aritméticos

Realizam operações matemáticas entre números.

- `+` soma
- `-` subtração
- `*` multiplicação
- `/` divisão
- `Mod` ou `%` resto da divisão

🔍 Relacionais

Comparam dois valores e retornam **verdadeiro** ou **falso**.

- `=` igual
- `<>` diferente
- `>` maior que
- `<` menor que
- `>=` maior ou igual
- `<=` menor ou igual

🔗 Lógicos

Combinam ou negam expressões lógicas para formar condições mais complexas.

- `e` — ambos verdadeiros
- `ou` — pelo menos um verdadeiro
- `nao` — inverte o valor lógico



Estruturas Condicionais — SE

A estrutura SE permite que o programa tome decisões: execute um bloco de comandos **se** uma condição for verdadeira, e outro bloco **senão**. É a base do raciocínio condicional na programação.

Sintaxe

```
se condição entao  
    comandos  
senao  
    comandos  
fimse
```

O bloco **senao** é opcional.
Use-o apenas quando
precisar executar algo
quando a condição for falsa.

Exemplo: Verificar Maioridade

```
se idade >= 18 entao  
    escreval("Maior de  
idade")  
senao  
    escreval("Menor de  
idade")  
fimse
```

Neste exemplo, o programa verifica se a variável **idade** é maior ou igual a 18. Caso seja, exibe "Maior de idade"; caso contrário, exibe "Menor de idade".

Estruturas de Repetição

Laços de repetição permitem que um bloco de comandos seja executado **múltiplas vezes**, evitando a duplicação de código. O Visualg oferece três estruturas distintas, cada uma com um caso de uso ideal.

1

enquanto ... faça

Testa a condição **antes** de executar. Se for falsa de início, nunca executa.

```
enquanto condição  
  faça  
    comandos  
fimenquanto
```

2

para ... de ... ate

Ideal quando se sabe **exatamente quantas vezes** repetir. O contador é incrementado automaticamente.

```
para i de 1 ate 10  
  faça  
    escreval(i)  
fimpara
```

3

repita ... ate

Testa a condição **depois** de executar. O bloco sempre roda pelo menos uma vez.

```
repita  
  comandos  
ate condição
```


Vetores, Procedimentos e Funções

Recursos avançados do Portugol que permitem organizar dados em coleções e reutilizar blocos de código, tornando os programas mais eficientes e legíveis.



Vetores (Arrays)

Um vetor armazena **vários valores do mesmo tipo** em uma única variável, acessados por índice.

```
var
  numeros: vetor[1..5]
de inteiro
inicio
  numeros[1] <- 10
  numeros[2] <- 25
  escreva(numeros[1])
```



Procedimento

Bloco de código **sem retorno**, usado para agrupar comandos reutilizáveis.

```
procedimento mensagem()
inicio
  escreva("Olá!")
fimprocedimento
```



Função

Bloco de código **com retorno**, que calcula e devolve um valor ao chamador.

```
funcao soma(a,b:
inteiro): inteiro
inicio
  retorne a + b
fimfuncao
```

Use **procedimentos** para ações (exibir mensagem, limpar tela) e **funções** para cálculos que precisam devolver um resultado.

Exemplo Completo: Par ou Ímpar

Este programa reúne os principais conceitos aprendidos: estrutura básica, variáveis, entrada/saída, operador `mod` e estrutura condicional. Leia o código com atenção e tente executá-lo no Visualg!

```
algoritmo "exemplo"
var
    numero: inteiro
inicio
    escreva("Digite um número: ")
    leia(numero)
    se numero mod 2 = 0 entao
        escreval("Número par")
    senao
        escreval("Número ímpar")
    fimse
fimalgoritmo
```

01 Declarar variável

`numero: inteiro` reserva espaço na memória para um número inteiro

02 Ler entrada

`leia(numero)` captura o valor digitado pelo usuário

03 Calcular resto

`numero mod 2` retorna 0 se for par, ou 1 se for ímpar

04 Exibir resultado

`escreval` mostra a mensagem correta conforme a condição